

RELATÓRIO DE IA - YASMIM

Pergunta: chat me responde se baseando no pdf do trabalho que anexe. como que eu posso organizar uma API em Node.js usando a separação entre routes, controllers, services, middlewares e database?

Resposta da IA: Com base no trabalho do Bolão da Copa 2026, a API em Node.js pode ser organizada em camadas para deixar o projeto mais limpo e fácil de entender. A pasta **routes** fica responsável por definir os caminhos da API, como **/api/login**, **/api/jogos** e **/api/palpites**. Já os **controllers** recebem a requisição do usuário e chamam a parte correta do sistema.

A pasta **services** concentra as regras de negócio, por exemplo validar um palpite, buscar dados externos ou verificar se o usuário pode alterar uma informação. Os **middlewares** servem para executar uma verificação antes da rota continuar, como conferir se o usuário enviou um token JWT válido. Por fim, a pasta **database** guarda a conexão com o banco, migrations e seeds, usando o Knex com SQLite.

Essa separação ajuda porque cada parte do projeto tem uma função específica. Assim, o código fica mais organizado, mais fácil de testar e mais fácil de corrigir caso apareça algum erro.

Eu: Depois da explicação, eu consegui entender que a API não deve ficar toda misturada em um único arquivo. O ideal é separar por responsabilidade: as rotas ficam só com os caminhos, os controllers recebem as requisições, os services cuidam das regras principais, os middlewares fazem verificações antes das rotas e a parte de database fica responsável pelo banco.

Pergunta: E qual é a diferença entre controller e service, e por que é melhor deixar a regra de negócio dentro do service?

Resposta da IA: A diferença entre **controller** e **service** é que o controller lida com a requisição e a resposta da API, enquanto o service cuida da regra de negócio.

No trabalho do Bolão da Copa 2026, por exemplo, o controller recebe uma requisição para criar um palpite, pega os dados enviados pelo usuário e chama o service. O service é quem faz as validações, como verificar se os gols são negativos, buscar informações externas como dólar e feriado, e salvar os dados no banco.

É melhor deixar a regra de negócio dentro do service porque o controller fica mais simples e organizado. Assim, se alguma regra precisar mudar, como a validação do palpite ou a forma de buscar dados externos, a alteração fica concentrada no service e não espalhada pelas rotas da API.

Eu: Eu entendi que o controller é mais como uma ponte entre a requisição e o resto do sistema. Ele recebe os dados, chama o service e devolve a resposta. Já o service é onde

ficam as regras mais importantes do projeto, como validar informações, buscar dados externos e salvar no banco

Pergunta: Como eu posso implementar login com JWT e usar o token para identificar o usuário logado, não entendi e não sei muito bem como faço me ajuda?

Resposta da IA: Para implementar login com JWT, primeiro o usuário envia o e-mail e a senha pela rota `POST /api/login`. O sistema procura esse usuário no banco de dados e compara a senha digitada com a senha salva, usando `bcrypt`.

Se os dados estiverem corretos, a API gera um token JWT. Esse token funciona como uma “chave de acesso” temporária. Dentro dele pode ser guardado o `id` do usuário, por exemplo:

```
const token = jwt.sign(  
  { id: usuario.id },  
  process.env.JWT_SECRET,  
  { expiresIn: '1d' }  
);
```

Depois disso, o usuário precisa enviar esse token nas rotas privadas, como criar, listar, alterar ou deletar palpites. O token é enviado no Postman na aba Authorization, usando o tipo `Bearer Token`.

No backend, o middleware de autenticação lê esse token, verifica se ele é válido e pega o `id` que foi salvo dentro dele. Assim, a API consegue saber qual usuário está logado sem precisar pedir e-mail e senha em toda requisição.

Exemplo:

```
const decoded = jwt.verify(token, process.env.JWT_SECRET);  
req.usuarioId = decoded.id;
```

Com isso, nas rotas de palpites, eu consigo usar `req.usuarioId` para salvar o palpite no nome do usuário logado e também para garantir que ele só veja, altere ou delete os próprios palpites.

Eu: Com a explicação, eu entendi que o JWT funciona como uma chave de acesso. Primeiro o usuário faz login com e-mail e senha, se estiver tudo certo, a API vai gerar um token com o id do usuário dentro. Depois, esse token usamos nas rotas privadas para o sistema saber quem está logado, sem precisar pedir a senha toda hora. Também entendi que esse id pode ser usado para salvar e buscar apenas os palpites daquele usuário.

Pergunta: E agora não sei como faço para criar um middleware de autenticação para bloquear rotas privadas quando o usuário não envia o token

Resposta da IA: Para bloquear rotas privadas, eu posso criar um middleware de autenticação. O middleware é uma função que roda antes da rota principal. No caso do trabalho, ele serve para verificar se o usuário enviou um token JWT válido.

O token normalmente vem no cabeçalho da requisição, no campo **Authorization**, no formato **Bearer token**. Primeiro o middleware verifica se esse token foi enviado. Se não foi, ele retorna erro **401**, informando que o token não foi informado.

Exemplo:

```
function authMiddleware(req, res, next) {
  const authHeader = req.headers.authorization;

  if (!authHeader) {
    return res.status(401).json({ mensagem: 'Token não informado' });
  }

  const token = authHeader.split(' ')[1];

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    req.userId = decoded.id;

    next();
  } catch (error) {
    return res.status(401).json({ mensagem: 'Token inválido' });
  }
}
```

Depois de criar o middleware, eu uso ele nas rotas privadas, como as rotas de palpites:

```
routes.post('/palpites', authMiddleware, palpitesController.criarPalpite);
routes.get('/palpites', authMiddleware, palpitesController.listarMeusPalpites);
routes.put('/palpites/:id', authMiddleware, palpitesController.atualizarPalpite);
routes.delete('/palpites/:id', authMiddleware, palpitesController.deletarPalpite);
```

Assim, se o usuário tentar acessar essas rotas sem token, a API bloqueia. Se o token for válido, o middleware deixa a requisição continuar usando o **next()**.

Eu: Depois da explicação, eu entendi que ele vai funcionar como uma verificação antes da rota principal. No caso da autenticação, ele vai conferir se o usuário mandou o token, se não mandou, a API bloqueia e retorna o erro. Se mandou um token válido, o middleware

deixa a rota continuar. Isso me ajudou a entender como proteger as rotas de palpites para que só usuários logados consigam acessar.

Pergunta: Tem alguma ideia de como eu posso fazer a rota de criação de palpite buscando informações externas, como dólar e feriados, usando `Promise.all`?

Resposta da IA: Para a rota de criação de palpite, eu posso usar `Promise.all` para buscar duas informações externas ao mesmo tempo: a cotação do dólar e a lista de feriados do ano do jogo.

A ideia é que, quando o usuário criar um palpite, ele envie dados como jogo, placar e data do jogo. A partir da data, eu pego o ano e uso esse ano para consultar a API de feriados. Ao mesmo tempo, consulto a API de cotação do dólar.

Exemplo:

```
const anoDoJogo = data_jogo.substring(0, 4);

const [respostaDolar, respostaFeriados] = await Promise.all([
  axios.get('https://economia.awesomeapi.com.br/json/last/USD-BRL'),
  axios.get(`https://brasilapi.com.br/api/feriados/v1/${anoDoJogo}`)
]);
```

O `Promise.all` é útil porque as duas requisições acontecem juntas, em vez de esperar uma terminar para começar a outra. Depois disso, eu consigo pegar o valor do dólar e verificar se a data do jogo é feriado.

```
const dolarNoDia = respostaDolar.data.USDBRL.bid;

const diaDeFeriado = respostaFeriados.data.some((feriado) => {
  return feriado.date === data_jogo;
});
```

Depois de buscar essas informações, eu salvo o palpite no banco junto com o `dolar_no_dia` e o `dia_de_feriado`. Assim, a rota não salva apenas o placar informado pelo usuário, mas também dados externos relacionados ao dia do jogo.

Eu: Com a explicação do promise, eu entendi que ele serve para fazer mais de uma busca ao mesmo tempo. No trabalho, isso vai ajudar porque na hora de criar um palpite a API precisa buscar o valor do dólar e também verificar os feriados. Em vez de fazer uma busca e depois a outra, o promise faz as duas juntas, deixando a rota mais rápido.